Сравнение собственного решения с аналогами

Вопросы комиссии уже в начале

- Зачем было разрабатывать собственную систему?
- Как задачу решали бы без вашего приложения?
- Какие аналоги уже существуют?
- Почему нельзя было использовать готовый сервис?
- Что ваше решение делает лучше, проще или точнее?

1. Что считать решением

Технологии — это реализация

Решение — это способ закрыть пользовательскую задачу.

Что НЕ является полным решением

«Я разработал сайт на React, Node.js и PostgreSQL».

Почему плохо:

- перечислены технологии
- не ясно, какую проблему решает система
- не видно пользователя и сценария
- нечего сравнивать с аналогами, кроме стека

Что считать веб-решением

Решение включает:

- проблему и целевую аудиторию
- пользовательские сценарии
- роли и права доступа
- данные и бизнес-процесс
- интерфейс и способ взаимодействия
- backend-логику и хранение данных
- ограничения, безопасность и поддержку

Пример

«Разработано веб-приложение для записи на консультации с использованием React и Express»

- *А зачем оно нужно?*
- *Почему нельзя использовать Google Forms?*
- *Что именно стало лучше?*
- *Для кого это решение?*

Пример

«Разработана веб-система записи студентов на консультации, которая автоматизирует выбор свободного времени, уведомление преподавателя и хранение истории записей».

+ *какая задача решается*
+ *кто пользователь*
+ *что можно сравнивать*
+ *где искать аналоги*

2. Что будет аналогом

Определение аналога

Аналог в ВКР — это любой другой способ решить ту же пользовательскую или организационную задачу.

Аналог не обязан быть сайтом с таким же стеком

Аналогом может быть ручной процесс, таблица, форма, готовый сервис, open-source проект или существующий продукт.

Типы аналогов

- Ручной процесс: звонок, бумага, сообщение, таблица
- Простое цифровое решение: Excel, Google Sheets, Google Forms
- Готовый сервис: SaaS, CRM, LMS, сервис бронирования
- Open-source решение
- Существующий коммерческий или учебный аналог
- Минимальная версия собственного решения

Baseline для веб-разработки

Baseline — это самый простой разумный способ решить задачу без вашей полноценной системы.

«Насколько моё приложение лучше простого варианта, который уже можно было бы использовать?»

Пример baseline

Задача: записывать студентов на консультации.

Baseline:

- преподаватель ведёт таблицу вручную;
- студент пишет в мессенджер;
- преподаватель подтверждает время;
- история хранится в переписке.

Если кажется, что аналогов нет

- Как задачу решали до приложения?
- Можно ли решить её таблицей/формой/txt?
- Есть ли готовый сервис с похожей функцией?
- Есть ли open-source проект?
- Какой MVP решает хотя бы 50% задачи?

3. Примеры аналогов

Пример 1: система записи

Решение: веб-система записи на консультации.

Аналоги:
- ручная запись через сообщения
- Google Forms + Google Sheets
- готовый сервис бронирования
- модуль записи в LMS
- собственная система с ролями и уведомлениями

Пример 2: интернет-магазин

Решение: интернет-магазин с каталогом, корзиной и личным кабинетом.

Аналоги:
- каталог товаров в социальной сети
- заказ через сообщения
- конструктор интернет-магазинов
- готовая CMS
- существующий магазин-конкурент
- собственная система

Пример 3: система управления задачами

Решение: веб-система для управления задачами команды.

Аналоги:
- Excel/Google Sheets
- Trello/YouGile/Jira
- open-source kanban-доска
- чат + список задач
- собственное решение под конкретный процесс

4. Критерии сравнения

Главный критерий

Не «какой стек современнее».

Главный вопрос:
«Насколько хорошо способ решает конкретную задачу конкретного пользователя?»

Технологии важны только тогда, когда они объясняют качество решения.

Группа 1: покрытие задачи

- Какие пользовательские сценарии поддерживаются?
- Какие роли предусмотрены?
- Какие требования закрыты полностью?
- Какие требования закрыты частично?
- Какие действия всё ещё выполняются вручную?
- Что не входит в границы ВКР?

Группа 2: удобство процесса

- Сколько шагов нужно пользователю?
- Сколько времени занимает сценарий?
- Нужно ли дублировать данные?
- Можно ли ошибиться при вводе?
- Насколько понятен интерфейс без инструкции?
- Что стало проще по сравнению с baseline?

Группа 3: практичность внедрения

- Нужно ли платить за внешний сервис?
- Можно ли развернуть систему локально?
- Кто администрирует систему?
- Как быстро добавить нового пользователя?
- Есть ли зависимость от сторонней платформы?
- Есть ли ограничения лицензии?

Группа 4: данные и интеграции

- Где хранятся данные?
- Кто контролирует доступ?
- Можно ли экспортировать данные?
- Есть ли интеграция с внешними сервисами?
- Что будет при недоступности внешнего API?
- Как обеспечивается целостность данных?

Группа 5: инженерные качества

- Производительность
- Надёжность
- Безопасность
- Масштабируемость
- Сопровождаемость
- Расширяемость
- Тестируемость

(Эти критерии должны быть связаны с задачей)

Плохие критерии

- «React лучше Vue»
- «PostgreSQL надёжнее MongoDB» без связи с задачей
- «Система удобная»
- «Есть личный кабинет» как доказательство качества
- «Сайт работает» как итоговое подтверждение
- «Использована современная архитектура»

Хорошие критерии

- Время выполнения основного сценария
- Количество действий пользователя
- Количество автоматизированных операций
- Покрытие требований
- Снижение ручной работы
- Контроль доступа к данным
- Наличие истории и статусов
- Возможность запуска и сопровождения

Что можно измерить

- Время выполнения сценария
- Количество кликов или экранов
- Количество ручных операций администратора
- Время ответа API
- Число реализованных требований
- Количество ошибок при тестовом сценарии
- Время запуска проекта по README

Аналог может оказаться лучше

- Готовый сервис может быть стабильнее
- SaaS может иметь больше функций
- Таблица может быть быстрее для маленькой группы
- CMS может быть дешевле разработки с нуля

Это не провал, если вы честно объясняете границы своего решения.

Вопросы к сильному аналогу

- Что он делает лучше вашего решения?
- Почему его нельзя просто использовать?
- Какие функции в нём избыточны?
- Какие ограничения есть у готового решения?
- Что в задаче требует собственной реализации?
- Что не стали бы реализовывать сознательно?

5. Честное сравнение

Что должно быть одинаковым

- Одна и та же пользовательская задача
- Один и тот же основной сценарий
- Одинаковый набор входных данных
- Похожие ограничения
- Одинаковые роли пользователей
- Одинаковый критерий оценки
- Честное описание того, что аналог не умеет

Один сценарий недостаточен

Если вы показали только успешный сценарий, это ещё не доказательство качества.

- *что будет при ошибочном вводе?*
- *что будет при отмене действия?*
- *что будет при конфликте данных?*
- *что будет при недоступности сервера?*
- *что будет у пользователя без прав?*

6. Как показать комиссии

Слайд сравнения: структура

- Строка: какая задача решается.
- **Таблица**: baseline, 1–2 аналога, своё решение.
- Критерии: 3–5 важных параметров.
- Вывод: в чём преимущество.
- Ограничение: где решение слабее или не проверено.
- Условие: какой сценарий сравнивался.

Пример: система онлайн-записи клиентов на услуги

Решение	Время оформления*	Действий администратора	Автоматиз. этапов	Настройка под процесс	Контроль над данными
Ручная запись	3–5 мин	5–7	0 из 4	вручную	вручную
Google Forms	2–3 мин	3–4	1 из 4	низкая	зависит от сервиса
Готовый сервис	~1 мин	0–1	4 из 4	средняя	ограничен тарифом
Своя система	~1 мин	0–1	4 из 4	высокая	полный

* 5 повторов типового сценария; в таблице указаны средние значения

Собственная система требует разработки и поддержки, но сокращает ручные операции и лучше адаптируется под требования конкретной организации.

Комиссия (оооочень) любит спрашивать

- Почему не использовать готовый сервис?
- Почему нельзя было решить задачу таблицей?
- Что именно автоматизировано?
- Что стало быстрее или проще?
- Какие функции аналога вы не реализовали?
- Где ваше решение слабее?
- Что является вашим личным вкладом?

Ответ комиссии: плохо / лучше

1. Готовый сервис не подходит, потому что мой проект уникальный

2. Готовый сервис закрывает базовую запись, но не поддерживает нужные роли, историю изменений и локальное хранение данных. Поэтому была разработана специализированная система под заданный процесс

Что является вкладом автора

- Анализ процесса и требований
- Проектирование пользовательских сценариев
- Реализация специализированной логики
- Схема данных и роли
- Автоматизация ручных операций
- Интеграции
- Тестирование сценариев
- Обоснование ограничений

7. Интерактивы

Найти baseline

Для каждой темы предложите baseline:
- интернет-магазин;
- система записи;
- личный кабинет студента;
- система заявок;
- образовательная платформа;
- CRM для небольшой организации.

Baseline должен решать задачу хотя бы грубо.

Выбрать критерии

Задача: система записи пациентов в клинику.

Какие критерии важнее?

- красивый интерфейс
- защита персональных данных
- предотвращение двойной записи
- скорость открытия главной страницы
- роли администратора, врача, уборщика, сантехника
- стоимость внедрения

Защитить слабое место

- нет мобильной версии
- нет интеграции с оплатой
- нет production deployment
- нет нагрузочного тестирования
- есть только базовые роли

почему это ограничение не уничтожает результат ВКР?

Не доказывайте, что ваш сайт лучше всех существующих систем.

Докажите, что разработанное решение разумно для конкретной задачи, пользователей, требований и ограничений.

Практическое задание

Подготовить один слайд/таблица: «Сравнение разработанного веб-решения с аналогами».

На слайде:
- задача
- baseline
- 1–2 аналога
- своё решение
- 3–5 критериев
- вывод
- ограничение